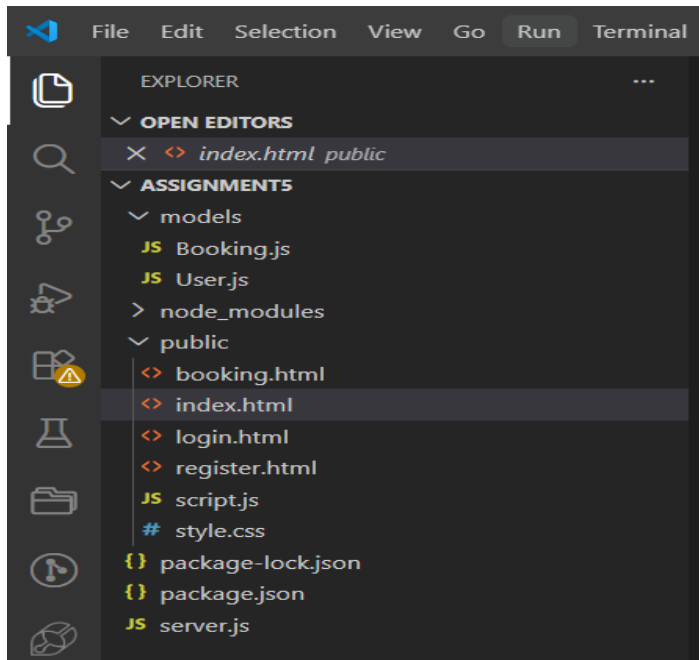


Assignment no 5

Assignment folder structure



Code:

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Travel Booking - Home</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header>
    <h1>Welcome to Travel Booking</h1>
  </header>
  <nav>
    <a href="index.html" class="active">Home</a>
    :
  </nav>
  <div class="container" style="max-width: 1100px;">
    <h2>Explore Our Best Packages</h2>
    <div class="packages">
      <div class="card">
```

```

        
        <div class="card-content">
            <h3>Goa Trip</h3>
            <p>3 Days, 2 Nights</p>
            <p style="font-size: 0.9rem; margin-top: 8px;">Experience the ultimate
beach getaway with beautiful sunsets.</p>
            <p class="price">₹5000</p>
        </div>
    </div>
<script src="script.js"></script>
<script>
    document.addEventListener('DOMContentLoaded', () => {
        if (localStorage.getItem('username')) {
            const prompt = document.getElementById('login-prompt');
            if (prompt) prompt.style.display = 'none';
        }
    });
</script>
</body>
</html>

```

Login

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width,
initial-scale=1.0">
    <title>Login - Travel Booking</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <h1>Travel Booking</h1>
    </header>
    <nav>
        <a href="index.html">Home</a>
        <a href="login.html" class="active">Login</a>
        <a href="register.html">Register</a>
    </nav>
    <div class="container">
        <h2>User Login</h2>
        <div id="msg" class="message"></div>
        <form id="loginForm"
onsubmit="handleLogin(event)">
            <label for="email">Email:</label>

```

Register

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-
width, initial-scale=1.0">
    <title>Register - Travel Booking</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
    <header>
        <h1>Travel Booking</h1>
    </header>
    <nav>
        <a href="index.html">Home</a>
        <a href="login.html">Login</a>
        <a href="register.html"
class="active">Register</a>
    </nav>
    <div class="container">
        <h2>User Registration</h2>
        <div id="msg" class="message"></div>
        <form id="registerForm"
onsubmit="handleRegister(event)">

```

<pre> <input type="email" id="email" required> <label for="password">Password:</label> <input type="password" id="password" required> <button type="submit">Login</button> <p class="text-center">Don't have an account? Register here</p> </form> </div> <script src="script.js"></script> </body> </html> </pre>	<pre> : <input type="password" id="password" required> <button type="submit">Create Account</button> <p class="text-center">Already have an account? Login here</p> </form> </div> <script src="script.js"></script> </body> </html> </pre>
--	--

SCRIPT.Js

```

function showMessage(elementId, text, type) {
  const msgElement = document.getElementById(elementId);
  msgElement.textContent = text;
  msgElement.className = `message ${type}`;
  msgElement.style.display = 'block';
  setTimeout(() => {
    msgElement.style.display = 'none';
  }, 3000);
}

async function handleRegister(e) {
  e.preventDefault();
  const name = document.getElementById('name').value;
  const email = document.getElementById('email').value;
  const password = document.getElementById('password').value;
  try {
    const response = await fetch('/register', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, email, password })
    });
    const data = await response.json();
    if (data.success) {
      showMessage('msg', data.message, 'success');
      document.getElementById('registerForm').reset();
      setTimeout(() => {
        window.location.href = 'login.html';
      }, 1000);
    } else {
      showMessage('msg', data.message, 'error');
    }
  }
}

```

```

    }
  } catch (err) {
    showMessage('msg', 'Error connecting to server', 'error');
  }
}

async function handleLogin(e) {
  e.preventDefault();
  const email = document.getElementById('email').value;
  const password = document.getElementById('password').value;
  try {
    const response = await fetch('/login', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ email, password })
    });
    const data = await response.json();
    if (data.success) {
      localStorage.setItem('username', data.username);
      showMessage('msg', data.message, 'success');
      setTimeout(() => {
        window.location.href = 'booking.html';
      }, 1000);
    } else {
      showMessage('msg', data.message, 'error');
    }
  } catch (err) {
    showMessage('msg', 'Error connecting to server', 'error');
  }
}

async function handleBooking(e) {
  e.preventDefault();
  const name = document.getElementById('b_name').value;
  const packageName = document.getElementById('b_package').value;
  const date = document.getElementById('b_date').value;
  try {
    const response = await fetch('/book', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ name, packageName, date })
    });
    const data = await response.json();
    if (data.success) {
      showMessage('msg', data.message, 'success');
      document.getElementById('bookingForm').reset();
      document.getElementById('b_name').value = localStorage.getItem('username');
    }
  }
}

```

```

        fetchBookings();
    } else {
        showMessage('msg', data.message, 'error');
    }
} catch (err) {
    showMessage('msg', 'Error connecting to server', 'error');
}
}
}
async function fetchBookings() {
    try {
        const response = await fetch('/bookings');
        const data = await response.json();
        if (data.success) {
            const list = document.getElementById('bookingsList');
            if(!list) return; // not on booking page
            list.innerHTML = "";

            if (data.bookings.length === 0) {
                list.innerHTML = '<tr><td colspan="3" class="text-center">No bookings
yet!</td></tr>';
                return;
            }
            data.bookings.forEach(b => {
                const tr = document.createElement('tr');
                tr.innerHTML = `
                    <td>${b.name}</td>
                    <td>${b.packageName}</td>
                    <td>${b.date}</td>
                `;
                list.appendChild(tr);
            });
        }
    } catch (err) {
        console.error('Error fetching bookings:', err);
    }
}
}
function checkAuthForNav() {
    const username = localStorage.getItem('username');
    if (username) {
        :
    }
}
function logout() {
    localStorage.removeItem('username');
    window.location.href = 'index.html';
}
document.addEventListener('DOMContentLoaded', checkAuthForNav);

```

Booking.js

```
const mongoose = require('mongoose');
const bookingSchema = new mongoose.Schema({
  name: { type: String, required: true },
  packageName: { type: String, required: true },
  date: { type: String, required: true }
});
module.exports = mongoose.model('Booking',
bookingSchema);
```

User.js

```
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true },
  password: { type: String, required: true } // Plain
text for simplicity in basic assignment
});
module.exports = mongoose.model('User',
userSchema);
```

SERVER.js

```
const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const path = require('path');
const app = express();
const PORT = 3005;
app.use(bodyParser.json());
app.use(bodyParser.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));
mongoose.connect('mongodb://127.0.0.1:27017/travelBookingDB')
  .then(() => console.log('Connected to MongoDB'))
  .catch(err => console.log('MongoDB connection error:', err));
const User = require('./models/User');
const Booking = require('./models/Booking');
app.post('/register', async (req, res) => {
  try {
    const { name, email, password } = req.body;
    const exists = await User.findOne({ email });
    if (exists) {
      return res.status(400).json({ success: false, message: 'Email already registered'
});
    }
    const newUser = new User({ name, email, password });
    await newUser.save();
    res.json({ success: true, message: 'Registration successful. Please login.' });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Server Error' });
  }
});
app.post('/login', async (req, res) => {
  try {
    const { email, password } = req.body;
    const user = await User.findOne({ email, password });
```

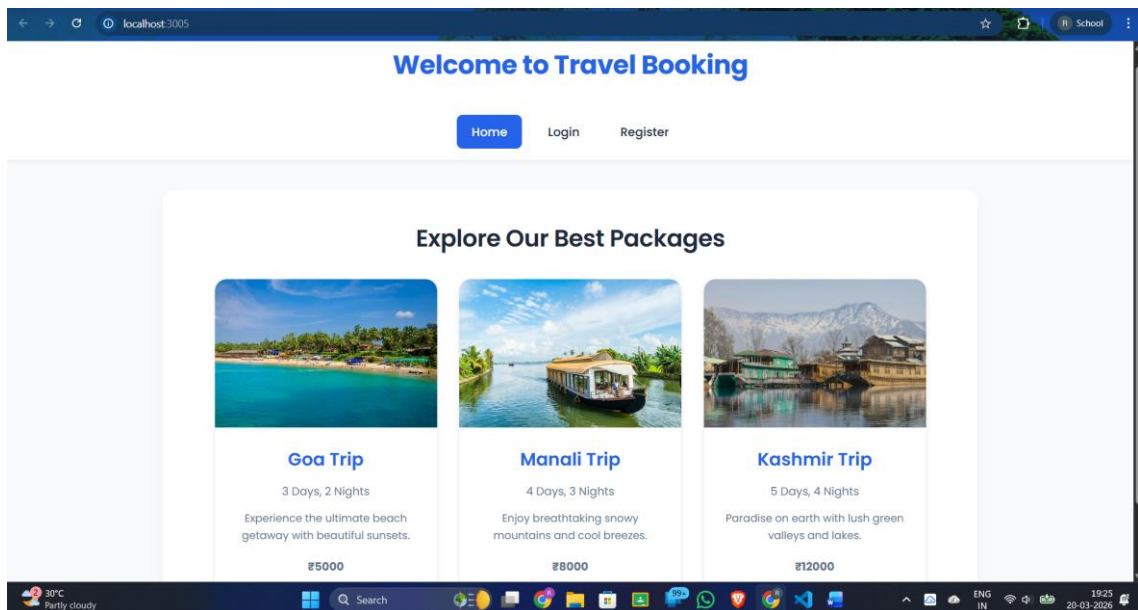
```

:
app.post('/book', async (req, res) => {
  try {
    const { name, packageName, date } = req.body;
    const newBooking = new Booking({ name, packageName, date });
    await newBooking.save();
    res.json({ success: true, message: 'Booking successful!' });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Server Error' });
  }
});
app.get('/bookings', async (req, res) => {
  try {
    const bookings = await Booking.find().sort({ _id: -1 });
    res.json({ success: true, bookings });
  } catch (err) {
    res.status(500).json({ success: false, message: 'Server Error' });
  }
});
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

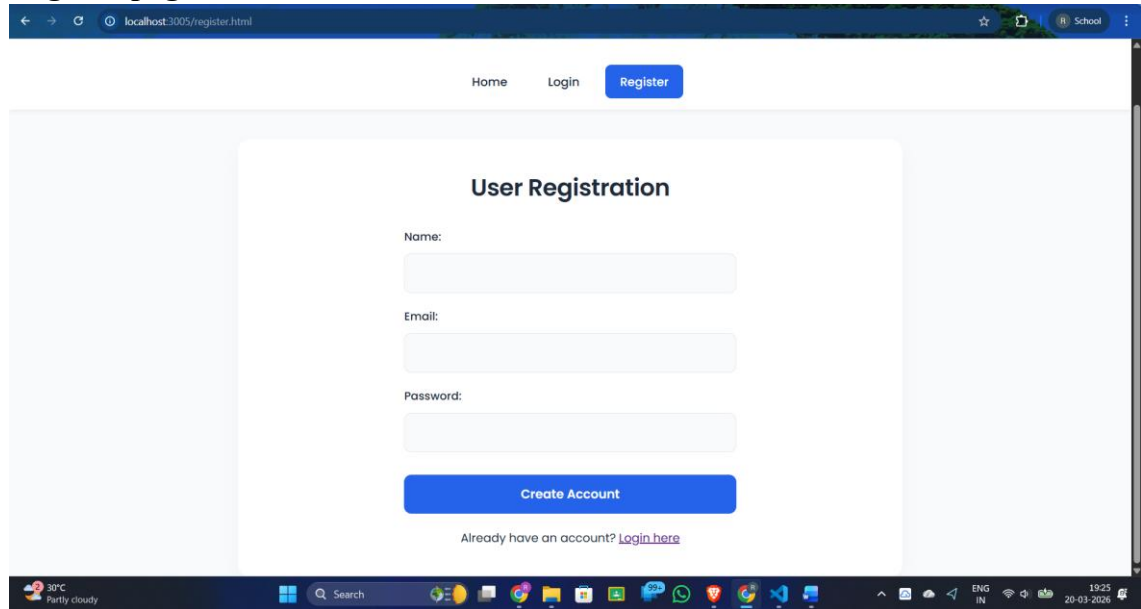
```

OUTPUT

Home page



Register page



The screenshot shows a web browser at localhost:3005/register.html. The page has a navigation bar with 'Home', 'Login', and a blue 'Register' button. The main content area is titled 'User Registration' and contains three input fields for 'Name:', 'Email:', and 'Password:'. Below these is a blue 'Create Account' button. At the bottom, there is a link: 'Already have an account? [Login here](#)'. The Windows taskbar at the bottom shows the date as 20-03-2020 and the time as 19:25.

Home Login Register

User Registration

Name:

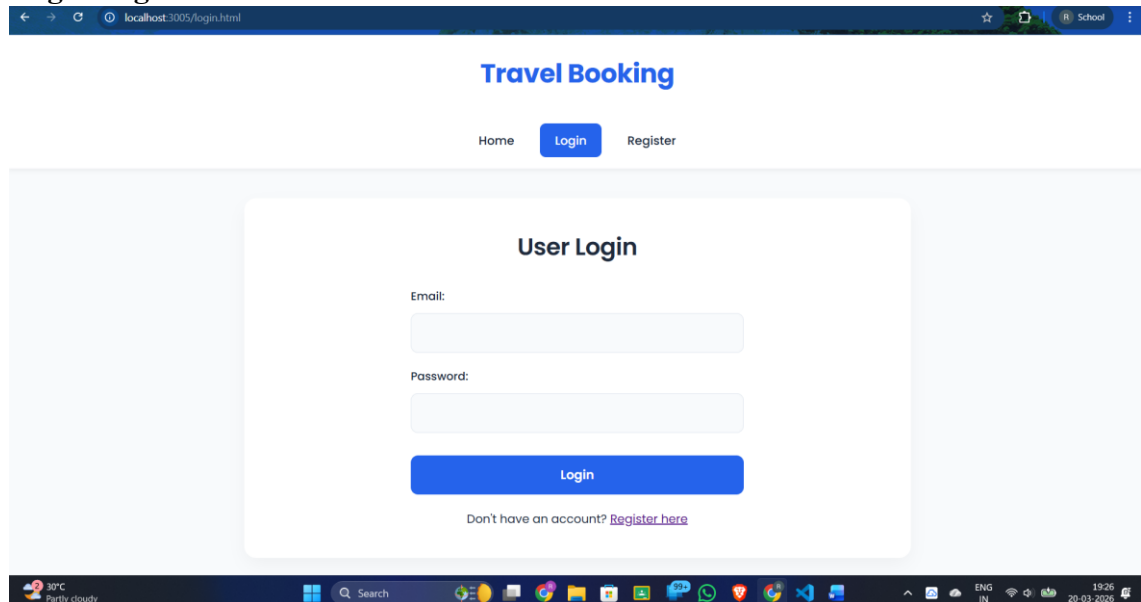
Email:

Password:

Create Account

Already have an account? [Login here](#)

Login Page



The screenshot shows a web browser at localhost:3005/login.html. The page has a navigation bar with 'Home', a blue 'Login' button, and 'Register'. The main content area is titled 'User Login' and contains two input fields for 'Email:' and 'Password:'. Below these is a blue 'Login' button. At the bottom, there is a link: 'Don't have an account? [Register here](#)'. The Windows taskbar at the bottom shows the date as 20-03-2020 and the time as 19:26.

Travel Booking

Home Login Register

User Login

Email:

Password:

Login

Don't have an account? [Register here](#)

Booking Page

[Home](#) [Booking page](#) [Logout](#)

Book a Travel Package

Welcome, **Ranjit**!

Your Name:

Select Package:

Travel Date:

[Confirm Booking](#)

All Bookings

Booking data

All Bookings		
Refresh List		
NAME	PACKAGE	DATE
Ranjit	Goa Trip - ₹5000	2026-03-27

Backend Booking storage

bookings

localhost:27017 > travelBookingDB > bookings

Documents 1 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

[EXPLAIN](#) [Reset](#) [Find](#) [Options](#)

[+ ADD DATA](#) [UPDATE](#) [DELETE](#) [EXPORT DATA](#) [EXPORT CODE](#)

25 1-1 of 1

```
{
  "_id": ObjectId('69bd526354d8a3b84873c68c'),
  "name": "Ranjit",
  "packageName": "Goa Trip - ₹5000",
  "date": "2026-03-27",
  "__v": 0
}
```

Backend user storage

users

+

localhost:27017 > travelBookingDB > users

Open MongoDB shell

Documents 2 Aggregations Schema Indexes 1 Validation

Type a query: { field: 'value' } or [Generate query](#)

Explain Reset Find Options

ADD DATA UPDATE DELETE EXPORT DATA EXPORT CODE

25 1 - 2 of 2

```
{
  "_id": ObjectId('69bcd32a9d2a06484695aeaf'),
  "name": "RANJIT CHAVAN",
  "email": "ranjitchavan11@gmail.com",
  "password": "Ranjit@7399",
  "__v": 0
}
```

```
{
  "_id": ObjectId('69bd524f54d0a3b84873c608'),
  "name": "Ranjit",
  "email": "ranjit.chavan23@pccoepune.org",
  "password": "Ranjit@7399",
  "__v": 0
}
```